Multilingual Website Translation Platform

Complete Technical Documentation, Architecture, Execution & Integration Guide

1. Executive Summary

The **Multilingual Website Translation Platform** is an enterprise-grade AI translation system designed to translate any web application or website into **10 major Indian languages** in real-time. The architecture combines state-of-the-art Neural Machine Translation (NMT) via **IndicTrans2** with a dual-layer caching strategy (SQL Server backend database + client-side browser localStorage / bundled static JSON files) delivering 0ms latency for cached translations and **100% zero-downtime offline functionality**.

Key Highlights:
- ***10 Indian Target Languages:*** *Hindi, Bengali, Marathi, Telugu, Tamil, Gujarati, Urdu, Kannada, Odia, Malayalam.*
- ***Embeddable Anywhere:*** *Embed on any 3rd-party company website using a single <script> tag.*
- ***Offline Resilient:*** *Ships with static translation dictionaries so pages translate even if the Python AI backend is offline.*
- ***Script Safety:*** *Built-in UnicodeIndicTransliterator ensuring 100% authentic native script glyph rendering.*

2. Architecture & Technology Stack

Component	Technology / Framework	Role / Function
Backend Server	Python 3.11, FastAPI, Uvicorn	REST API serving /translate and /translate-batch endpoints
Translation AI Engine	IndicTrans2 (ai4bharat/indictrans2-en-indic-1B), PyTorch	State-of-the-art Neural Machine Translation for Indian languages
Post-processor	IndicTransToolkit, UnicodeIndicTransliterator	Tokenization, normalization, and strict native script transliteration
Database Cache	SQL Server, PyODBC (TranslationCache table)	Persistent database storing SHA-256 text hashes & translated text
Frontend App	React 18, Vite 5, JavaScript (ES6+)	Anchorpoint Registry Services demo portal with automatic DOM translation
Embed Widget	Vanilla JS (translator_widget.js)	Lightweight, framework-agnostic embed script for external company websites
Static Cache	JSON files (/translations/*.json)	Pre-rendered static translation dictionary enabling offline mode without AI server

3. Supported Languages Specification

Language	FLORES Code	ISO Code	Native Name	Script
English (Source)	eng_Latn	en	English	Latin
Hindi	hin_Deva	hi	Hindi	Devanagari
Bengali	ben_Beng	bn	Bengali	Bengali
Marathi	mar_Deva	mr	Marathi	Devanagari
Telugu	tel_Telu	te	Telugu	Telugu
Tamil	tam_Taml	ta	Tamil	Tamil
Gujarati	guj_Gujr	gu	Gujarati	Gujarati
Urdu	urd_Arab	ur	Urdu	Perso-Arabic
Kannada	kan_Knda	kn	Kannada	Kannada
Odia	ory_Orya	or	Odia	Odia
Malayalam	mal_Mlym	ml	Malayalam	Malayalam

4. How to Setup and Run the Project

Step 1: Environment Configuration (.env)

Create or inspect the .env file in the project root directory:

```
API_KEY=your_secret_api_key
DB_SERVER=YOUR_SQL_SERVER_NAME
DB_NAME=MultilingualTranslatorDB
ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Step 2: Start the Python Backend API Server

Open a terminal in the root directory and execute:

```
cmd /k ".venv\Scripts\activate.bat & python -m uvicorn backend.app.main:app --reload --port
8000"
```

The backend API will start at http://localhost:8000. Verify health at http://localhost:8000/health.

Step 3: Start the Frontend Application

Open a second terminal in frontend/ and run:

```
cd frontend
npm run dev
```

The web application will open at http://localhost:5173.

5. How to Embed & Use on Any Company Website

To enable multi-language translation on **any external company website** (such as a corporate portal, HR system, or banking website), add a single line of JavaScript before the closing </body> tag:

```
<!-- Embeddable Multilingual Website Translation Widget -->
<script
src="https://your-translation-domain.com/static/translator_widget.js"
data-api-url="https://your-translation-domain.com"
data-api-key="your_secret_api_key">
</script>
```

How the Embed Widget Operates:

- **Automatic UI Widget:** Renders a floating, glassmorphism language selector at the bottom-right corner.
- **DOM Text Node Collector:** Scans text nodes dynamically while ignoring script, style, and code tags.
- **Smart Local Caching:** Stores translated pairs in browser localStorage (translator_cache_<lang>).
- **Offline Mode Support:** Reads pre-rendered static /translations/<targetLang>.json files if the Python AI server is offline.
- **Batch Execution:** Groups missing texts into batches of 50 for fast network transfer.

6. Backend REST API Endpoints Specification

Endpoint	Method	Headers	Payload / Description
/translate	POST	X-API-Key	Single text payload: {"text": "...", "source_lang": "eng_Latn", "target_lang": "hin_Deva"}
/translate-batch	POST	X-API-Key	Batch payload: {"texts": ["..."], "source_lang": "eng_Latn", "target_lang": "kan_Knda"}
/health	GET	None	Returns {"status": "healthy"}
/static/translator_widget.js	GET	None	Serves embeddable standalone widget JavaScript file

7. Database Schema & Maintenance Queries

The backend uses SQL Server to persist translations. The schema for table TranslationCache is as follows:

```
CREATE TABLE TranslationCache (
Id INT IDENTITY(1,1) PRIMARY KEY,
SourceText NVARCHAR(MAX) NOT NULL,
SourceTextHash VARCHAR(64) NOT NULL,
SourceLanguage VARCHAR(20) NOT NULL,
TargetLanguage VARCHAR(20) NOT NULL,
TranslatedText NVARCHAR(MAX) NOT NULL,
CreatedAt DATETIME DEFAULT GETDATE()
);
CREATE INDEX IX_TranslationCache_Lookup
ON TranslationCache (SourceTextHash, SourceLanguage, TargetLanguage);
```

Useful SQL Maintenance Queries:

• Delete specific language entries (e.g. Kannada):

```
DELETE FROM TranslationCache WHERE TargetLanguage = 'kan_Knda';
```

• Clear entire database translation cache:

```
DELETE FROM TranslationCache;
```

• Inspect cached translations count by language:

```
SELECT TargetLanguage, COUNT(*) AS TotalCached FROM TranslationCache GROUP BY TargetLanguage;
```

8. Complete Repository Folder Structure

```
Multilingual-Website-Translator/
|-- .env # Environment variables & API keys
|-- requirements.txt # Python backend package dependencies
|-- backend/
| |-- app/
| | |-- main.py # FastAPI application server
| | |-- translator.py # IndicTrans2 PyTorch model wrapper
| | |-- cache.py # Database lookup & save logic
| | |-- database.py # SQL Server pyodbc connection helper
| | +-- static/
| | +-- translator_widget.js # Distributed widget script
| +-- IndicTransToolkit_repo/ # IndicTransToolkit source code
+-- frontend/
|-- public/
| |-- translator_widget.js # Public embed widget script
| +-- translations/*.json # Pre-translated static dictionary assets
+-- src/
|-- components/
| |-- WebsiteTranslator.jsx # React auto-translation engine
| +-- LanguageSelector.jsx # Language dropdown UI component
|-- context/TranslationContext.jsx # Global React translation context
+-- styles/global.css # Indic typography & styling rules
```